UNIVERSITI MALAYA

NFMN

SIM 1005 / SIN 1002

Fundamentals of Spreadsheets / Introduction to Worksheet

Lecture 9: VBA – Constructs and Macro Elements

By

Assoc. Prof. Dr Noor Fadiya Mohd Noor

Institute of Mathematical Sciences

Objectives

- To execute commands in VBA using conditional statements and loops.
- To debug errors in Macro.
- To manipulate Macro elements (strings, date/time, events and arrays) in VBA

Original notes by NFM Noor

VBA Continues..

- VBA includes:
  6) If Then Statement
  7) Loop
  8) Macro Errors
  9) String Manipulation
  10) Date and Time
  11) Events
  12) Array

Original notes by NFM Noor

If Then Statement

- Used to execute a command when a specific condition is met.
- There are 2 statements:
  1) If-Then
  2) If-Then-Else-End-If

Original notes by NFM Noor

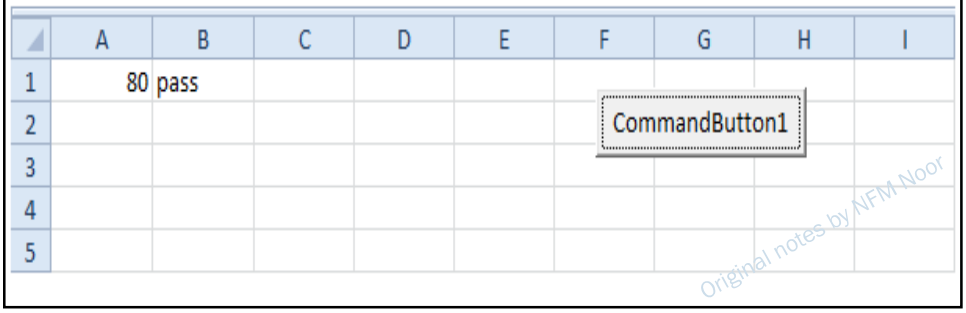
If Then Statement

Place a command button on your worksheet and add the following code lines:

```
Dim score As Integer, result As String
score = Range("A1").Value

If score >= 60 Then result = "pass"

Range("B1").Value = result
```



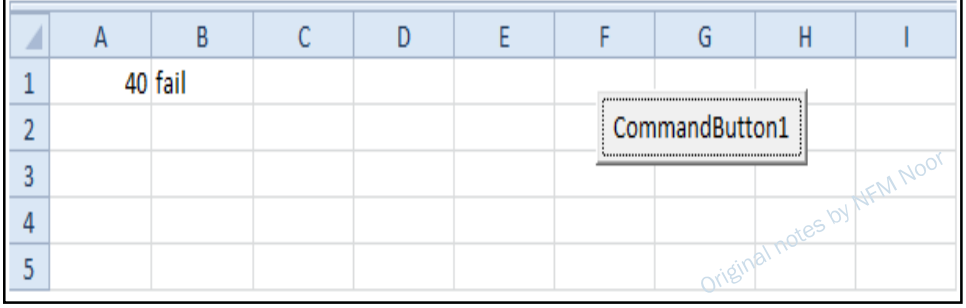
Else Statement

Place a command button on your worksheet and add the following code lines:

```
Dim score As Integer, result As String
score = Range("A1").Value

If score >= 60 Then
    result = "pass"
Else
    result = "fail"
End If

Range("B1").Value = result
```



Loop

- Used to execute a command repetitively as long as a specific condition is met.
- Types of Loop:
  1) Single For-Next Loop
  2) Double For-Next Loop
  3) Triple For-Next Loop
  4) Do-While Loop

Original notes by NFM Noor

Single Loop

You can use a single loop to loop through a one-dimensional range of cells.

Place a command button on your worksheet and add the following code lines:

```
Dim i As Integer

For i = 1 To 6
    Cells(i, 1).Value = 100
Next i
```

	A	B	C	D	E	F	G	H	I
1	100								
2	100								
3	100								
4	100								
5	100								
6	100								
7									
8									

CommandButton1

Original notes by NFM Noor

Double Loop

You can use a double loop to loop through a two-dimensional range of cells.

Place a command button on your worksheet and add the following code lines:

```
Dim i As Integer, j As Integer

For i = 1 To 6
    For j = 1 To 2
        Cells(i, j).Value = 100
    Next j
Next i
```

	A	B	C	D	E	F	G	H	I
1	100	100							
2	100	100							
3	100	100							
4	100	100							
5	100	100							
6	100	100							
7									
8									

CommandButton1

Original notes by NFM Noor

Triple Loop

You can use a triple loop to loop through two-dimensional ranges on multiple Excel worksheets.

Place a command button on your worksheet and add the following code lines:

```
Dim c As Integer, i As Integer, j As Integer

For c = 1 To 3
    For i = 1 To 6
        For j = 1 To 2
            Worksheets(c).Cells(i, j).Value = 100
        Next j
    Next i
Next c
```

Explanation: The only change made compared to the code for the double loop is that we have added one more loop and added Worksheets(c). in front of Cells to get the two-dimensional range on the first sheet for c = 1, the second sheet for c = 2 and the third sheet for c = 3. Download the Excel file to see this result.

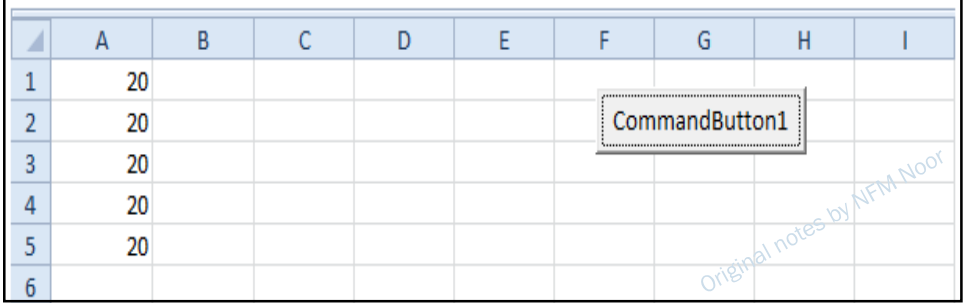
Original notes by NFM Noor

Do While Loop

1. Place a command button on your worksheet and add the following code lines:

```
Dim i As Integer
i = 1

Do While i < 6
    Cells(i, 1).Value = 20
    i = i + 1
Loop
```



Original notes by NFM Noor

2. Enter some numbers in column A.

	A	B	C	D	E	F	G	H	I
1	27								
2	35								
3	56								
4	59								
5	85								
6	84								
7									
8									

3. Place a command button on your worksheet and add the following code lines:

```
Dim i As Integer
i = 1

Do While Cells(i, 1).Value <> ""
    Cells(i, 2).Value = Cells(i, 1).Value + 10
    i = i + 1
Loop
```

Original notes by NFM Noor

Result when you click the command button on the sheet:

	A	B	C	D	E	F	G	H	I
1	27	37							
2	35	45							
3	56	66							
4	59	69							
5	85	95							
6	84	94							
7									
8									

CommandButton1

Original notes by NFM Noor

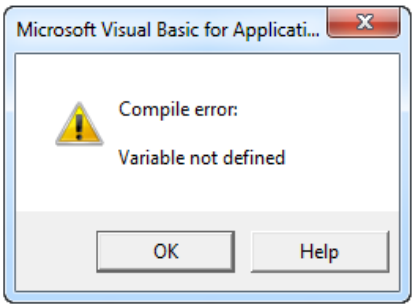
Macro Errors

Place a command button on your worksheet and add the following code lines:

```
x = 2
Range("A1").Valu = x
```

1. Click the command button on the sheet.

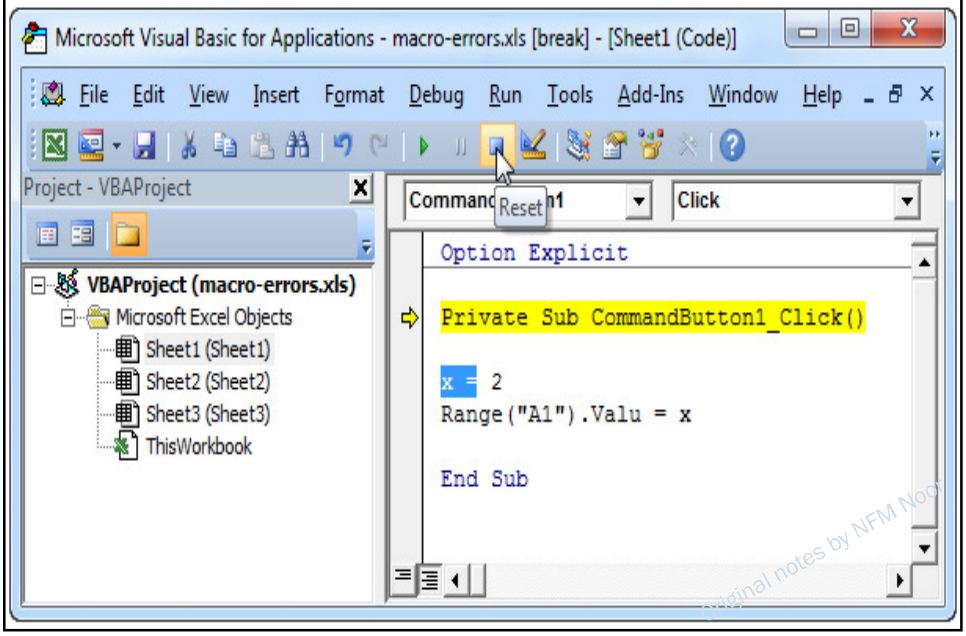
Result:



Original notes by NFM Noor

2. Click OK.

3. In the Visual Basic Editor, click Reset to stop the debugger.



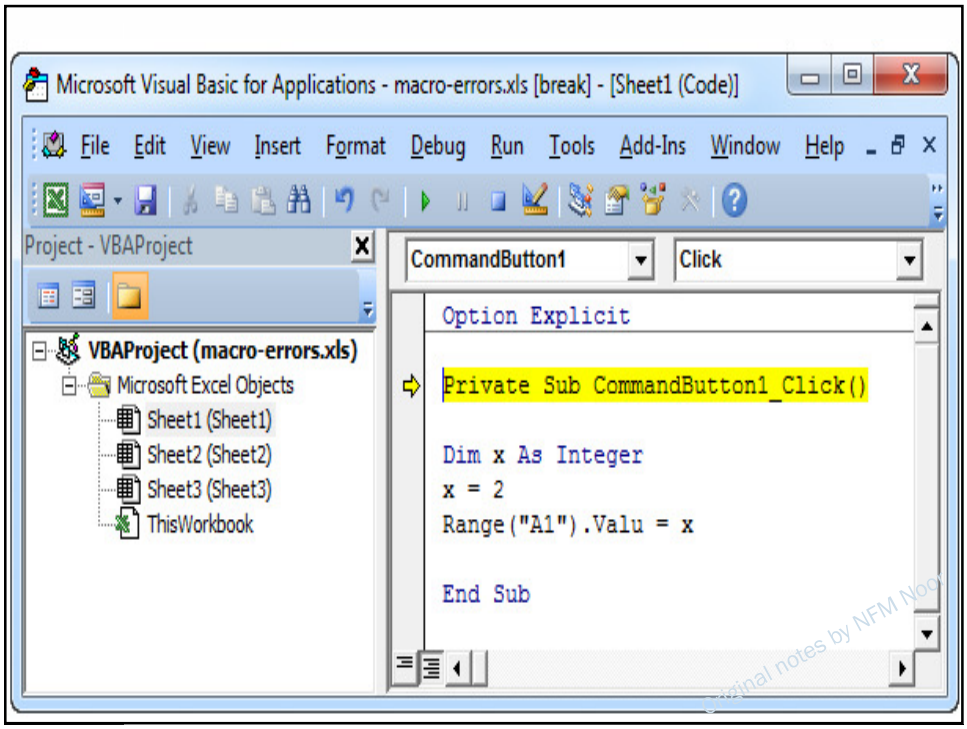
4. Correct the error by adding the following code line at the start of the code.

```
Dim x As Integer
```

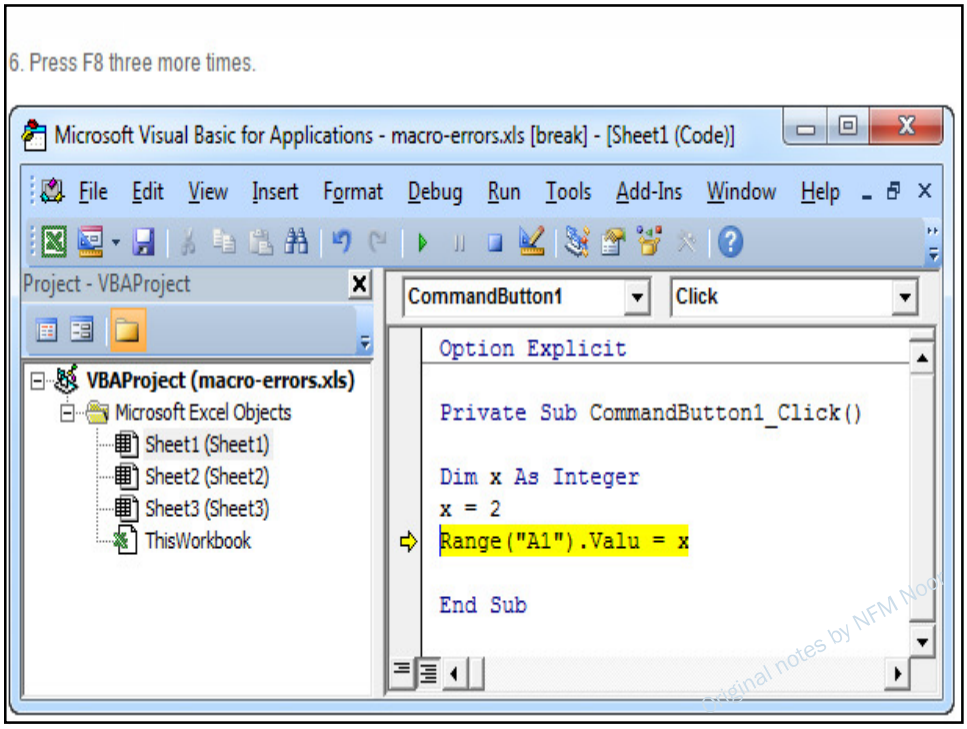
You may have heard of the technique called debugging before. With this technique you can step through your code.

5. In the Visual Basic Editor, place your cursor before Private and press F8.

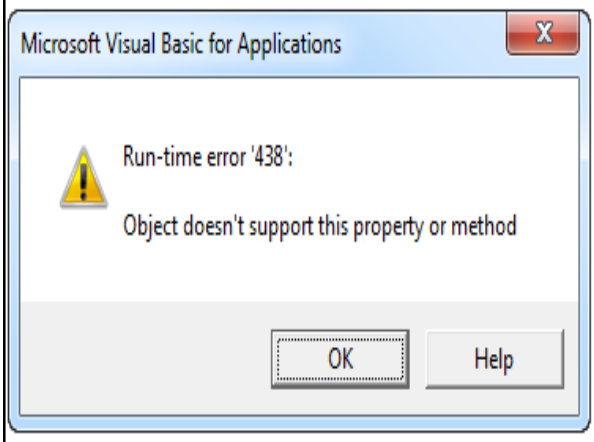
The first line turns yellow.



6. Press F8 three more times.



The following error appears.



The Range object has a property called Value. Value isn't spelled correctly here. Debugging is a great way to not only find errors, but also understand code better. Our Debugging example program shows you how to single step through your code and see the effect of each code line on your worksheet.

Original notes by NFM Noor

String Manipulation

Join Strings

We use the & operator to concatenate (join) strings.

Code:

```
Dim text1 As String, text2 As String
text1 = "Hi"
text2 = "Tim"

MsgBox text1 & " " & text2
```

Result:



Original notes by NFM Noor

Left

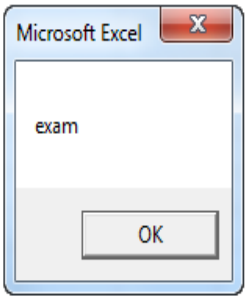
To extract the leftmost characters from a string, use Left.

Code:

```
Dim text As String
text = "example text"

MsgBox Left(text, 4)
```

Result:



Original notes by NFM Noor

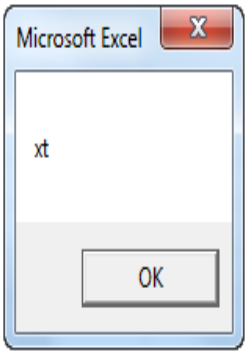
Right

To extract the rightmost characters from a string, use Right. We can also directly insert text in a function.

Code:

```
MsgBox Right("example text", 2)
```

Result:



Original notes by NFM Noor

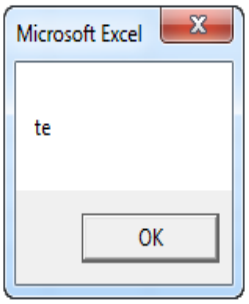
Mid

To extract a substring, starting in the middle of a string, use Mid.

Code:

```
MsgBox Mid("example text", 9, 2)
```

Result:



Note: started at position 9 (t) with length 2. You can omit the third argument if you want to extract a substring starting in the middle of a string, until the end of the string.

Original notes by NFM Noor

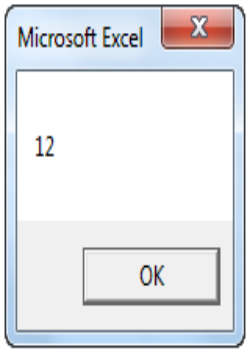
Len

To get the length of a string, use Len.

Code:

```
MsgBox Len("example text")
```

Result:



Original notes by NFM Noor

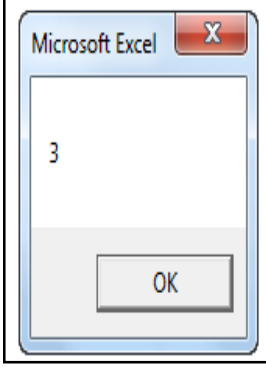
Instr

To find the position of a substring in a string, use Instr.

Code:

```
MsgBox Instr("example text", "am")
```

Result:



Original notes by NFM Noor

Date and Time

Year, Month, Day of a Date

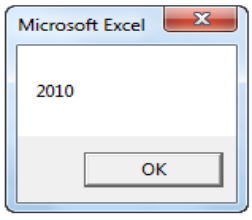
The following macro gets the year of a date. To declare a date, use the Dim statement. To initialize a date, use the DateValue function.

Code:

```
Dim exampleDate As Date

exampleDate = DateValue("Jun 19, 2010")

MsgBox Year(exampleDate)
```



Original notes by NFM Noor

Note: Use Month and Day to get the month and day of a date.

DateAdd

To add a number of days to a date, use the DateAdd function. The DateAdd function has three arguments. Fill in "d" for the first argument to add days. Fill in 3 for the second argument to add 3 days. The third argument represents the date to which the number of days will be added.

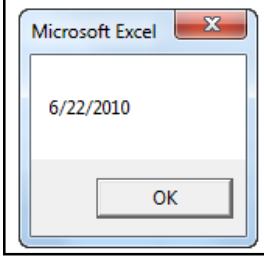
Code:

```
Dim firstDate As Date, secondDate As Date

firstDate = DateValue("Jun 19, 2010")
secondDate = DateAdd("d", 3, firstDate)

MsgBox secondDate
```

Result:



Original notes by NFM Noor

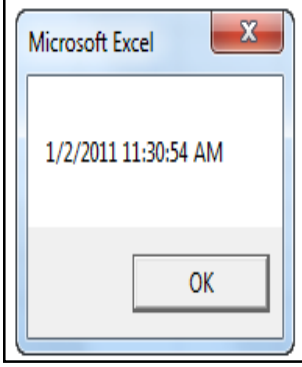
Current Date and Time

To get the current date and time, use the Now function.

Code:

```
MsgBox Now
```

Result:



Original notes by NFM Noor

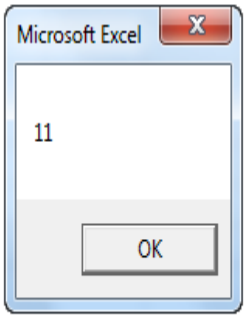
Hour, Minute, Second

The get the hour of a time, use the Hour function.

Code:

```
MsgBox Hour(Now)
```

Result:



Note: Use Minute and Second to get the minute and second of a time.

Original notes by NFM Noor

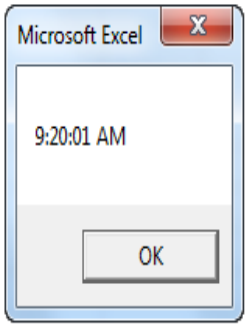
TimeValue

The TimeValue function converts a string to a time serial number. The time's serial number is a number between 0 and 1. For example, noon (halfway through the day) is represented as 0.5.

Code:

```
MsgBox TimeValue("9:20:01 am")
```

Result:

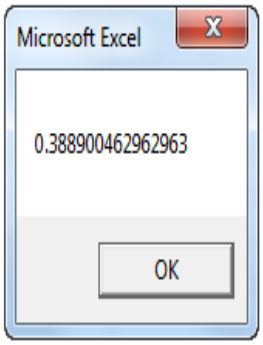


Original notes by NFM Noor

Now, to clearly see that Excel handles times internally as numbers between 0 and 1, add the following code lines:

```
Dim y As Double
y = TimeValue("09:20:01")
MsgBox y
```

Result:



Original notes by NFM Noor

Events

- Actions performed by user that trigger Excel VBA to execute code.

Workbook Open Event

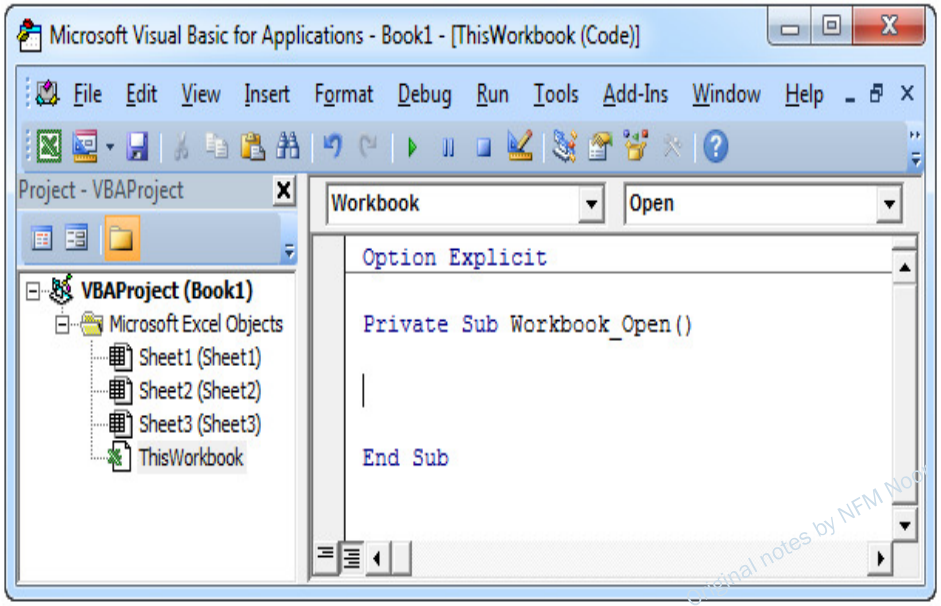
Code added to the Workbook Open Event will be executed by Excel VBA when you open the workbook.

1. Open the Visual Basic Editor.

2. Double click on This Workbook in the Project Explorer.

Original notes by NFM Noor

3. Choose Workbook from the left drop-down list. Choose Open from the right drop-down list.

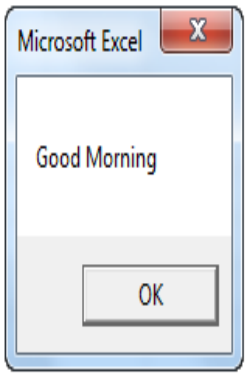


4. Add the following code line to the Workbook Open Event:

```
MsgBox "Good Morning"
```

5. Save, close and reopen the Excel file.

Result:



Original notes by NFM Noor

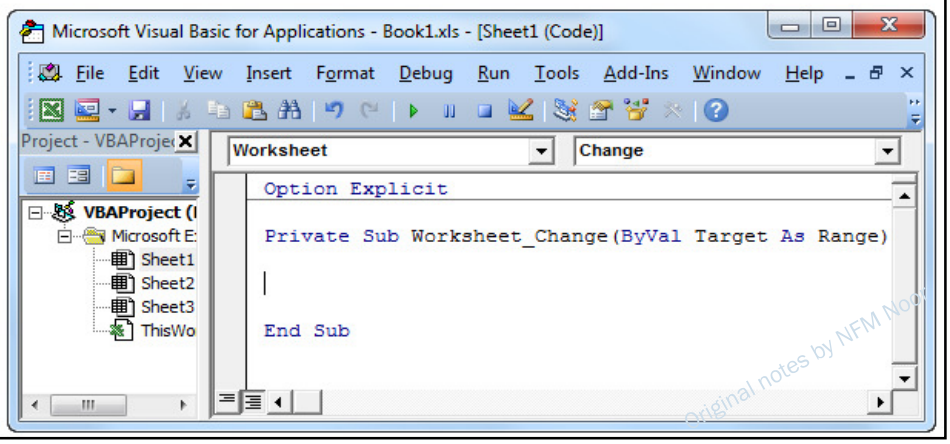
Worksheet Change Event

Code added to the Worksheet Change Event will be executed by Excel VBA when you change a cell on a worksheet.

1. Open the Visual Basic Editor.

2. Double click on a sheet (for example Sheet1) in the Project Explorer.

3. Choose Worksheet from the left drop-down list. Choose Change from the right drop-down list.



Add the following code lines to the Worksheet Change Event:

4. The Worksheet Change Event listens to all changes on Sheet1. We only want Excel VBA to do something if something changes in cell B2. To achieve this, add the following code lines:

```
If Target.Address = "$B$2" Then


End If
```

5. We only want Excel VBA to show a MsgBox if the user enters a value greater than 80. To achieve this, add the following code line between If and End If.

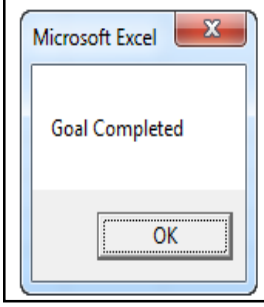
```
If Target.Value > 80 Then MsgBox "Goal Completed"
```

Original notes by NFM Noor

6. On Sheet1, enter a number greater than 80 into cell B2.

	A	B	C	D	E	F	G	H	I
1									
2		81							
3									
4									
5									

Result:



Original notes by NFM Noor

Array

One-dimensional Array

To create a one-dimensional array, execute the following steps.

Place a command button on your worksheet and add the following code lines:

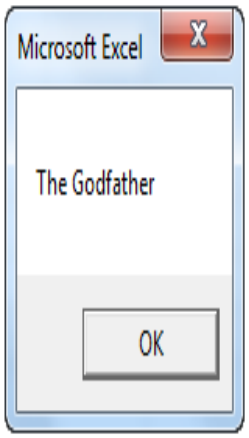
```
Dim Films(1 To 5) As String

Films(1) = "Lord of the Rings"
Films(2) = "Speed"
Films(3) = "Star Wars"
Films(4) = "The Godfather"
Films(5) = "Pulp Fiction"

MsgBox Films(4)
```

Original notes by NFM Noor

Result when you click the command button on the sheet:



Explanation: the first code line declares a String array with name Films. The array consists of five elements. Next, we initialize each element of the array. Finally, we display the fourth element using a MsgBox.

Original notes by NFM Noor

Two-dimensional Array

To create a two-dimensional array, execute the following steps. This time we are going to read the names from the sheet.

L17

	A	B	C	D	E	F	G	H
1	Lord of the Rings	Adventure						
2	Speed	Action						
3	Star Wars	Sci-Fi						
4	The Godfather	Crime						
5	Pulp Fiction	Drama						
6								
7								

CommandButton1

Place a command button on your worksheet and add the following code lines:

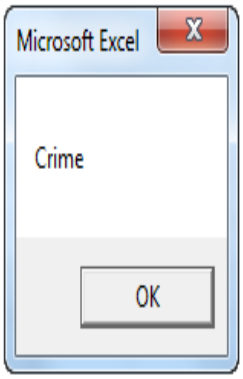
```
Dim Films(1 To 5, 1 To 2) As String
Dim i As Integer, j As Integer

For i = 1 To 5
    For j = 1 To 2
        Films(i, j) = Cells(i, j).Value
    Next j
Next i

MsgBox Films(4, 2)
```

Original notes by NFM Noor

Result when you click the command button on the sheet:



Explanation: the first code line declares a String array with name Films. The array has two dimensions. It consists of 5 rows and 2 columns. Tip: rows go first, then columns. The other two variables of type Integer are used for the Double Loop to initialize each element of the array. Finally, we display the element at the intersection of row 4 and column 2.

Original notes by NFM Noor

Summary

- Excel is a powerful spreadsheet to automate tasks involving constructs, time/date, events and arrays.
- If-Then and If-Then-Else-End-If ensures a command is executed once when specific conditions are met.
- Loop ensures a command is executed repetitively when specific conditions are met.
- Debugging is a vital process in programming to debug errors.
- Good manipulation of macro elements (strings, date/time, events, arrays) is a key for a successful VBA project.

Original notes by NFM Noor